

Experiment 2

Name: Yug Panchal

Roll No: 62

Class: Btech 9

Academic Year: 2025–26

Experiment No.2

Title

MLP Model for Non linearly separable Logic Gates

Problem Statement

MLP model for Binary Non Linearly Separable Logic Gates

Aim

Implement and evaluate a MLP model to simulate Non Linearly Separable binary logic gates using fundamental neural network principles.

Source Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def plot_mlp_decision_boundary(xor_inputs, xor_outputs, hidden_weights,
                               hidden_bias, output_weights, output_bias, sigmoid, title):
    plt.figure(figsize=(8, 7))
    plt.title(title)
    plt.xlabel('Input 1')
    plt.ylabel('Input 2')
    plt.xlim(-0.5, 1.5)
    plt.ylim(-0.5, 1.5)
    plt.grid(True)
    x_min, x_max = -0.5, 1.5
```

```

y_min, y_max = -0.5, 1.5

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                      np.linspace(y_min, y_max, 200))

grid_points = np.c_[xx.ravel(), yy.ravel()]

hidden_layer_input_grid = np.dot(grid_points, hidden_weights) +
hidden_bias

hidden_layer_output_grid = sigmoid(hidden_layer_input_grid)

output_layer_input_grid = np.dot(hidden_layer_output_grid,
output_weights) + output_bias

predicted_output_grid = sigmoid(output_layer_input_grid)

Z = (predicted_output_grid > 0.5).astype(int).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)

for i in range(len(xor_inputs)):
    input_data = xor_inputs[i]

    expected_output = xor_outputs[i][0]

    color = 'red' if expected_output == 0 else 'blue'

    marker = 'o' if expected_output == 0 else 's'

    plt.scatter(input_data[0], input_data[1], marker=marker,
color=color, s=100, edgecolor='black', zorder=5,

                label=f'Class {expected_output}' if i == 0 or i == 1
else "") # Add label once per class

from matplotlib.lines import Line2D

legend_elements = [

    Line2D([0], [0], marker='o', color='w', label='Output: 0',
markerfacecolor='red', markersize=10, markeredgcolor='black'),

    Line2D([0], [0], marker='s', color='w', label='Output: 1',
markerfacecolor='blue', markersize=10, markeredgcolor='black')

]

plt.legend(handles=legend_elements, loc='upper left')

plt.show()

```

```

xor_inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
xor_outputs = np.array([[0], [1], [1], [0]])

# Sigmoid activation function and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Neural network parameters
input_size = 2
hidden_size = 2 # At least two hidden neurons for XOR
output_size = 1
learning_rate = 0.1
epochs = 10000

# Initialize weights and biases randomly
np.random.seed(42) # for reproducibility
hidden_weights = np.random.uniform(size=(input_size, hidden_size))
hidden_bias = np.random.uniform(size=(1, hidden_size))
output_weights = np.random.uniform(size=(hidden_size, output_size))
output_bias = np.random.uniform(size=(1, output_size))

print("Initial Hidden Weights:\n", hidden_weights)
print("Initial Hidden Bias:\n", hidden_bias)
print("Initial Output Weights:\n", output_weights)
print("Initial Output Bias:\n", output_bias)

# Training the MLP

```

```

print("\nTraining the MLP for XOR...")
for epoch in range(epochs):
    # Forward Propagation

    hidden_layer_input = np.dot(xor_inputs, hidden_weights) + hidden_bias
    hidden_layer_output = sigmoid(hidden_layer_input)

    output_layer_input = np.dot(hidden_layer_output, output_weights) +
output_bias
    predicted_output = sigmoid(output_layer_input)

    # Backpropagation
    # Calculate error for output layer
    output_error = xor_outputs - predicted_output
    output_delta = output_error * sigmoid_derivative(predicted_output)

    # Calculate error for hidden layer
    hidden_error = output_delta.dot(output_weights.T)
    hidden_delta = hidden_error * sigmoid_derivative(hidden_layer_output)

    # Update weights and biases
    output_weights += hidden_layer_output.T.dot(output_delta) *
learning_rate
    output_bias += np.sum(output_delta, axis=0, keepdims=True) *
learning_rate

    hidden_weights += xor_inputs.T.dot(hidden_delta) * learning_rate
    hidden_bias += np.sum(hidden_delta, axis=0, keepdims=True) *
learning_rate

    if epoch % 1000 == 0:
        loss = np.mean(np.square(xor_outputs - predicted_output))
        print(f"Epoch {epoch}, Loss: {loss:.4f}")

```

```

print("\nMLP training complete.")
print("\nFinal Hidden Weights:\n", hidden_weights)
print("Final Hidden Bias:\n", hidden_bias)
print("Final Output Weights:\n", output_weights)
print("Final Output Bias:\n", output_bias)

# Test the trained MLP
print("\nTesting the trained MLP for XOR:")
predictions = []
for i, input_data in enumerate(xor_inputs):
    hidden_layer_input = np.dot(input_data, hidden_weights) + hidden_bias
    hidden_layer_output = sigmoid(hidden_layer_input)
    output_layer_input = np.dot(hidden_layer_output, output_weights) +
output_bias
    predicted_val = sigmoid(output_layer_input)
    predicted_class = 1 if predicted_val[0] > 0.5 else 0
    predictions.append({'Input': input_data.tolist(), 'Predicted Output':
predicted_class, 'Expected Output': xor_outputs[i][0]})
    print(f"Input: {input_data}, Predicted: {predicted_class}, Expected:
{xor_outputs[i][0]}")

xor_df = pd.DataFrame(predictions)
print("\nXOR Gate Truth Table (DataFrame with MLP predictions):")
print(xor_df.to_markdown(index=False))

# Verify accuracy
actual_xor_outputs = xor_df['Predicted Output'].tolist()
expected_xor_outputs = xor_df['Expected Output'].tolist()

if actual_xor_outputs == expected_xor_outputs:

```

```

    print("\nVerification: XOR gate MLP implementation is CORRECT.")
else:
    print("\nVerification: XOR gate MLP implementation is INCORRECT.
Predicted:", actual_xor_outputs, "Expected:", expected_xor_outputs)

# Visualize the XOR MLP decision boundary
plot_mlp_decision_boundary(xor_inputs, xor_outputs, hidden_weights,
hidden_bias, output_weights, output_bias, sigmoid, 'XOR Gate MLP Decision
Boundary')

```

Results Obtained:

```

Initial Hidden Weights:
[[0.37454012 0.95071431]
 [0.73199394 0.59865848]]
Initial Hidden Bias:
[[0.15601864 0.15599452]]
Initial Output Weights:
[[0.05808361]
 [0.86617615]]
Initial Output Bias:
[[0.60111501]]

Training the MLP for XOR...
Epoch 0, Loss: 0.3247
Epoch 1000, Loss: 0.2406
Epoch 2000, Loss: 0.1960
Epoch 3000, Loss: 0.1207
Epoch 4000, Loss: 0.0305
Epoch 5000, Loss: 0.0125
Epoch 6000, Loss: 0.0074
Epoch 7000, Loss: 0.0051
Epoch 8000, Loss: 0.0038
Epoch 9000, Loss: 0.0031

MLP training complete.

Final Hidden Weights:
[[3.79198478 5.81661184]
 [3.80004873 5.8545897 ]]
Final Hidden Bias:
[[-5.82020057 -2.46277158]]
Final Output Weights:
[[-8.32186051]
 [ 7.66063503]]
Final Output Bias:
[[-3.45550373]]

Testing the trained MLP for XOR:
Input: [0 0], Predicted: 0, Expected: 0
Input: [0 1], Predicted: 1, Expected: 1
Input: [1 0], Predicted: 1, Expected: 1
Input: [1 1], Predicted: 0, Expected: 0

```

XOR Gate Truth Table (DataFrame with MLP predictions):

Input	Predicted Output	Expected Output
[0, 0]	0	0
[0, 1]	1	1
[1, 0]	1	1
[1, 1]	0	0

Verification: XOR gate MLP implementation is CORRECT.

